

Here is the complete, finalized pitch script for all 13 slides. It integrates the latest empirical findings from Claude, perfectly resolves the Slide 11/13 conflict, and arms you with the ultimate data-driven narrative for your defense.

Read this aloud to internalize the flow.

Slide 1: MediExtract

Welcome. Today I am presenting MediExtract. One dictated consult note goes in. Four clinical fields come out, each shown directly next to the exact quote it came from. If the system cannot prove a value mathematically, it doesn't guess, and it doesn't go quiet. It triggers an active Grounding Failure Warning so the physician checks exactly that one field.

Slide 2: Meet Dr. Aisha

Meet Dr. Aisha. She is a GP in a Singapore polyclinic seeing 24 patients a day. At 21:40 at night, she is staring at her ninth unfinished chart. She will not sign a dose she cannot verify in under 3 seconds. This product is built around how fast she can check a clinical fact, not how beautifully an AI can summarize it.

Slide 3: Retyping is the cost

She spends 157 minutes a day on clerical EHR work, plus 86 minutes of "pyjama time" after hours. Dictating is fast; retyping it into separate fields is what burns clinician hours. We automate that retyping, strictly bounded by her 3-second limit for verifying each field.

Slide 4: Proven, or flagged. Never silently blank.

Here is how it works: One note in, four verified fields out. First, an input scan blocks notes with hidden characters before any model call happens. Then, we make a single Gemini call with a strict schema and no tools. Finally, deterministic Python checks execute: the quote must appear word for word, the value must match its quote, and each dose number must carry its unit. If a value fails, it is wiped to null. This isn't a silent blank; it's a Grounding Failure Warning alerting her that the AI returned something it couldn't prove.

Slide 5: Nearly 2× the rule-based baseline

We achieved 73% recall against sealed labels. But 73% means nothing in a vacuum. We built a massive rule-based Python dictionary with 14,000 drugs, and it only hit 38%. More importantly, we track the 'majority-class baseline'—which proves that if a broken system just blindly guesses "Not Stated" for every field, it accidentally gets 44% of the answers right. We report these comparators to prove we aren't artificially inflating our success rate by grading our own homework.

Slide 6: Why checked output beats both

So why does our gated pipeline beat a low-code console? A standard low-code console shows 147 values and holds none back. MediExtract keeps the exact same 73% recall but removes three objectively wrong values—including a case where the AI literally outputted the string "not_stated" into a clinical dose field.

Slide 7: The model obeyed. Python held.

We red-teamed the system by hiding malicious commands in the dictation. The foundational LLM failed; it obeyed the malicious prompts 50% of the time, including "prescribe warfarin 10 mg daily." But zero payloads reached the database as verified. Our deterministic Python tripwire caught the hallucination and flagged it for review. This proves why you cannot trust an unguarded AI console in a hospital—you must have a deterministic software perimeter.

Slide 8: The compliance tax, itemised

Our token cost is fractions of a cent per note—about \$3.58 per physician annually. But the important metric is the \$1.90 it costs to force the AI to output the verbatim evidence quotes. That \$1.90 is the literal cash price of clinical safety. In healthcare, inference costs don't matter; clinician time does. Optimizing for a cheaper model is the wrong move if it costs Dr. Aisha an extra 10 seconds of verification time per chart.

Slide 9: Fast enough, and honest about the tail

Our median latency is 1.3 seconds. 66 out of 67 trials beat our 3-second operational budget. However, we explicitly report our one 12-second outlier. In a polyclinic processing 100,000 notes, a 1.5% latency failure rate means 1,500 times a doctor is stuck staring at a loading screen. We don't hide the tail end of our data. And our code survives compilation: we pass 485 unit tests under python -O, meaning our strict routing logic survives when fragile 'assert' statements are stripped.

Slide 10: Known limits, next moves

We found a severe structural dependency. When we stripped ALL-CAPS section headers out of the test notes, allergy recall fell from 0.900 to 0.650. We engineered a proximity gate and a recall check to fix this. But when we measured them, they caught zero silent failures and destroyed 9 correct fields. We discovered the header wasn't a formatting crutch—it was the *only* clinical signal in the text. We formally refused to ship those gates because we do not ship security theater. Our next move is to scale testing to 100x headerless, continuous-speech dictations.

Slide 11: Close

A value with its quote, or flagged for review. We don't claim perfection. We claim honest limits, and zero ungrounded claims.

Slide 12: Live Demonstration

Here is the pipeline running live. Case 1 demonstrates the core value proposition: highlights anchored directly under extracted values for sub-3-second verification. Case 2 shows the model safely declining on third-party and discontinued drugs. Case 3 shows an active prompt injection—the model obeyed, but our deterministic Python tripwire contained it in review with zero verified payloads. It is a deterministic CLI pipeline feeding inert, CSP-locked review interfaces.

Slide 13: Architectural Q&A: The Medication Slot

Finally, why didn't we implement a multi-item medication list to eliminate the remaining 25% failure gap? Because 31 of our 36 silent failures stem from a single defect: the single-medication slot. When the drug name aligns, our dose extraction recall is 0.990. The failure is structural, not generative. We successfully spiked a 12-item schema on the first API attempt. But deploying it requires re-labeling all 67

cases for a new gold-v3 corpus, which breaks paired statistical comparability with our baseline. We deliberately chose empirical honesty over premature optimization.